

PySeA

Python Scripted e-Assessment for creating ONYX-tests

27. Netzwerktreffen Mathematik/Physik + E-Learning - 17. September 2026

Julian Schmidt, Kjell Bühler, **Felix Kaule**, Anke Bucher, Gerlind Schubert

Motivation: Herausforderungen mit der grafischen Benutzeroberfläche von ONYX

- Die Erstellung über die **grafische Oberfläche** ist manchmal **langsam** und **umständlich**, besonders in **komplexen, aufeinander aufbauenden Aufgaben**:

Bilder min. 6 Klicks

Quelle* Keine ausgewählt

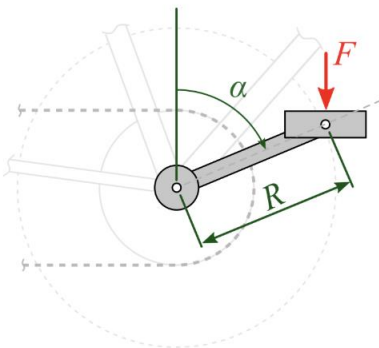
Alternativtext

Größe Breite Höhe

☒ Seitenverhältnis beibehalten

Vorschau

Bei nachfolgendem Bild handelt es sich um eine Vorschau, daher kann die angeze



Feedback > 7 Klicks

Titel

Bedingungen* Punkte Lücke 3

Lösungsversuche

Verknüpfung der Bedingungen ☒ UND ☐ ODER

Feedback*

Bearbeiten Format Tabelle Werkzeuge Hilfe

B *I* U

<>

Das Moment berechnet sich zu:

$$M = \sin(\alpha) \cdot F \cdot R = \sin(\alpha) \cdot \{F\} \cdot \{R\}$$

$$\{F\} \cdot \{R\} = \{M\}$$

P

Motivation: Herausforderungen mit der grafischen Benutzeroberfläche von ONYX

- Die Erstellung über die **grafische Oberfläche** ist manchmal **langsam** und **umständlich**, besonders in **komplexen, aufeinander aufbauenden Aufgaben**:

Variantensteuerung

Aufwändig und unübersichtlich

Matrikelnummer
✓ Erreicht: 1 von 1 Punkt(en)

Bitte geben Sie Ihre Matrikelnummer für die Variablenzuweisung ein:

Matrikelnummer: 12345 ✓ (0)

Hinweis: Achten Sie darauf keine Leerzeichen zu verwenden.

Vielen Dank!

Ihre Matrikelnummer lautet: 12345

Verwenden Sie für die Bearbeitung Ihres Belegs folgende Parameter:

$a = 1 \text{ m}$

$b = 4,5 \text{ m}$

🔄 Frage noch einmal beantworten

Frage 1/34

Parametersets für Gruppen von Studierenden basierend auf der Eingabe der Matrikelnummer

Parameter

Übergabe in Unteraufgaben umständlich und unübersichtlich

Übergabe der Parameter / Verknüpfungen

{ gk1 }	Gleitkommazahl	Berechnung (MAXIMA): {HEA450}	Lastwerte - {gk1}, Stab 1-3 (pL) - {gk1V}, Stab 1-3 (IL) - {gk1par}
{ gk2 }	Gleitkommazahl	Berechnung (MAXIMA): {HEA400}	Lastwerte - {gk2}, Stab 1-3 (pL) - {gk2V}, Stab 1-3 (IL) - {gk2par}
{ gk3 }	Gleitkommazahl	Berechnung (MAXIMA): {HEB600}+{decke}*{b}	Lastwerte - {gk3}, Stab 1-3 (pL) - {gk3V}, Stab 1-3 (IL) - {gk3ort}
{ gk4 }	Gleitkommazahl	Berechnung (MAXIMA): {PE360}+{dach}*{b}	Lastwerte - {gk4}

Name { gk3 }

Typ

Anzeige formatieren ☐

Wertsatzung auf

{HEB600}+{decke}*{b}

Zur Angabe Ihrer Berechnungsvorschrift nutzen Sie die Schreibweise des Computer-Algebra-Systems MAXIMA, bspw. "float((a)*(b)/5)".

Verknüpfungen

Lastwerte	gk3
Stab 1-3 (pL)	gk3V
Stab 1-3 (IL)	gk3ort

Setzen Sie den Ausgangswert von Variablen einzelner Aufgaben auf den aktuellen Wert dieser Testvariable. Der Ausgangswert wird vor der Ausführung aller Variablenberechnung in der Aufgabe selbst gesetzt.

Verknüpfung hinzufügen

Motivation: Herausforderungen mit der grafischen Benutzeroberfläche von ONYX

- Die Erstellung über die **grafische Oberfläche** ist manchmal **langsam** und **umständlich**, besonders in **komplexen, aufeinander aufbauenden Aufgaben**:

Laufende Tests

Zur Fehlerbehebung ein Neustart des Tests für alle Studierenden

Titel und Beschreibung

Sichtbarkeit

Zugang

Test-Konfiguration

Variablen

Bewertung

Mehrsprachigkeit

Zugeordneter Test

Aktuelle Auswahl: Aufgabenpool/Archiv/TM Festigkeitslehre/Tests/Schubbeanspruchungen 2024_korrigiert

Version: v1, erstellt am 05.07.2024 um 16:27 Uhr

Letzte Änderung: am 17.06.2025 um 17:21 Uhr

ID der Ressource: 45370900536

Austauschen

~~Editieren~~

PySeA: Was ist das?

- Eine von uns entwickelte Python-Bibliothek, um außerhalb der GUI von ONYX Aufgaben und Tests zu erstellen

Python Skript Minimalbeispiel

```
from backend.facades import TestFacade

test = TestFacade("Minimalbeispiel")

sektion = test.add_section("Rechnen")
aufgabe = sektion.add_task("Addition")

loesung = aufgabe.response(4)
aufgabe.item_body("Berechnen Sie: $$2 + 2 =$$ " + loesung)

test.create_test("examples/output")
```

ONYX Test

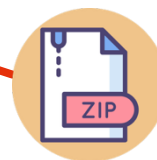
Addition

Punkte: 1

Keine Antwort

Berechnen Sie: 2 + 2 =

Antwort abgeben



Test als Zip-Datei (QTI)

Der ONYX-Test kann dann
wie gewohnt weiter bearbeitet
werden

PySeA: Was ist das?

Python Skript mit Bild und Feedback

```
from backend.facades import TestFacade

test = TestFacade("Bildbeispiel")

bilder = test.load_images("examples/assets").map()
bild_Taschenrechner = bilder["Taschenrechner.png"]
bild_Taschenrechner.set_width("200")

sektion = test.add_section("Rechnen")
aufgabe = sektion.add_task("Addition")

loesung = aufgabe.response(4)
aufgabe.item_body(
    "Berechnen Sie: $$2 + 2 =$$ " + loesung + "\n" +
    bild_Taschenrechner)

aufgabe.feedback("correct", "Richtig")
aufgabe.feedback("incorrect", "Falsch")

test.create_test("examples/output")
```

ONYX Test

Addition

⊗ Erreicht: 0 von 1 Punkt(en)

Berechnen Sie: $2 + 2 =$ ❌ (4)



Falsch!

Falsch

PySeA: Funktionsumfang

Aufgabentypen

Auswahl (Single, Multiple)
Einfache Zuordnung
Grafische Zuordnung
Numerische Lücken

Varianten

Zuweisung je Matrikelnummer
oder Anzahl der Varianten

Exceldatei Import

Nutzung als Tabelle, Lösung
oder Parametervariante

Bilder

Ordner laden, per Name nutzen
Größe im Skript
in Text, Optionen, Feedback

Feedback

richtig / falsch
ab Versuch n, Versuchsbereich
mit Text, Formel und Bild

Formeln

LaTeX mit $\\$\\dots\\$$
in Aufgabe und Optionen
im Feedback

Einstellungen

Toleranz absolut / relativ /
exakt
Punkte, Abzug je Versuch
Navigation, Bestehensgrenze

Export

QTI-Zip für ONYX und OPAL
JSON zum Weiterbearbeiten

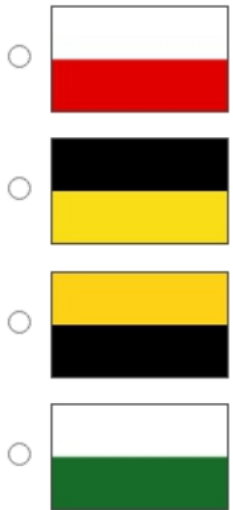
Das Ergebnis ist ein normaler ONYX-Test, nach dem Import wie gewohnt im Editor bearbeitbar.

PySeA Aufgabentypen: Auswahlaufgabe

Auswahlaufgabe

Mit der Auswahlaufgabe geben Sie mehrere Antwortmöglichkeiten vor, aus denen der Lernende die richtigen Antworten selektieren muss. Sie können zwischen einer einfachen Auswahl (Single Choice) und einer mehrfachen Auswahl (Multiple Choice) wählen.

Das Bundesland Sachsen hat die folgende Flagge...



```
from backend.facades import TestFacade
```

```
test = TestFacade("Auswahlaufgabe")  
sektion = test.add_section("Sachsen")  
aufgabe = sektion.add_task("Flagge")
```

```
flaggen = test.load_images("flaggen").map()  
auswahl = aufgabe.selection("singleChoice")  
auswahl.set_correct([flaggen["sachsen.png"]])  
auswahl.set_incorrect([flaggen["polen.png"],  
                        flaggen["schwarz_gelb.png"],  
                        flaggen["gelb_schwarz.png"]])
```

```
aufgabe.item_body("Das Bundesland Sachsen hat "  
                 "die folgende Flagge...\n" + auswahl)
```

```
test.create_test("output")
```

PySeA Aufgabentypen: Einfache Zuordnung

Einfache Zuordnung (Drag-and-Drop)

Eine einfache Zuordnungsaufgabe erlaubt die Zuordnung von Antwortpaaren zueinander. Dabei können immer nur zwei Elemente per Drag and Drop einander zugeordnet werden.

Der wichtigste, größte und einzig schiffbare Fluss des Bundeslandes Sachsens ist die Elbe. Die verschiedenen **Quellflüsse der Elbe** durchziehen die sächsischen Städte. Ordne anhand des Flusslaufs die Quellflüsse den gegebenen Städten zu.

Weißeritz	Korrektes Element hier ablegen	Dresden
Saale	Korrektes Element hier ablegen	Bautzen
Mulde	Korrektes Element hier ablegen	Halle
Spree	Korrektes Element hier ablegen	Leipzig

```
from backend.facades import TestFacade
```

```
test = TestFacade("Einfache Zuordnung")  
sektion = test.add_section("Sachsen")  
aufgabe = sektion.add_task("Flüsse")
```

```
zuordnung = aufgabe.matching()  
zuordnung.add_pair("Weißeritz", "Dresden")  
zuordnung.add_pair("Saale", "Halle")  
zuordnung.add_pair("Mulde", "Leipzig")  
zuordnung.add_pair("Spree", "Bautzen")
```

```
aufgabe.item_body(  
    "Die Quellflüsse der Elbe durchziehen die "  
    "sächsischen Städte. Ordne die Quellflüsse "  
    "den gegebenen Städten zu.\n" + zuordnung)
```

```
test.create_test("output")
```

PySeA Aufgabentypen: Grafische Zuordnung

Grafische Zuordnung

Die grafische Zuordnungsaufgabe ermöglicht die Platzierung von Elementen auf vorgegebene Bereiche einer Hintergrundgrafik. Es wird genau ein Element einem Bereich zugeordnet.

Wo liegen Sachsens Großstädte?

Platzieren Sie die gegebenen Städte an der richtigen Position auf der Karte.



Chemnitz

Leipzig

Dresden

```
from backend.facades import TestFacade
```

```
test = TestFacade("Grafische Zuordnung")  
sektion = test.add_section("Sachsen")  
aufgabe = sektion.add_task("Großstädte")
```

```
karte = aufgabe.graphical_assignment()  
karte.original_image_path = "karte/Original.png"  
karte.mask_image_path = "karte/Maske.png"  
karte.correct_image_path = "karte/Richtig.png"  
karte.add_wrong_images(["karte/Falsch.png"])
```

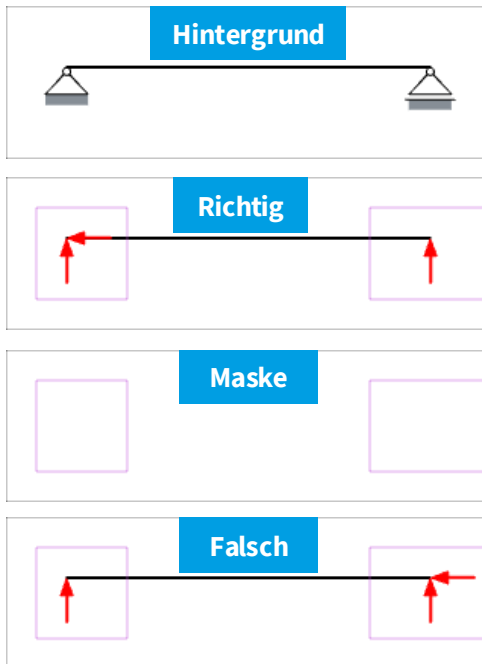
```
aufgabe.item_body(  
    "Wo liegen Sachsens Großstädte?\n"  
    "Platzieren Sie die gegebenen Städte an der "  
    "richtigen Position auf der Karte.\n" + karte)
```

```
test.create_test("output")
```

PySeA Aufgabentypen: Grafische Zuordnung

Grundidee

- Das System wurde entwickelt, um eine exakte Zuordnung hinsichtlich Position und Größe der Bilder zu ermöglichen (händisch ist das sehr mühsam).



Grafische Zuordnungsaufgabe

Ordnen Sie die Bildausschnitte den richtigen Bereichen im Hintergrundbild zu:



Antwort abgeben

PySeA Aufgabentypen: Numerische Eingabe

Numerische Eingabe

Die numerische Aufgabe ist eine Lückentextaufgabe speziell für numerische Eingaben. Sie können absolute und relative Toleranzen festlegen und so einen Lösungsbereich definieren.

Das Bundesland Sachsen grenzt an andere
deutsche Bundesländer und Länder außerhalb
Deutschlands.

```
from backend.facades import TestFacade

test = TestFacade("Numerische Eingabe")
sektion = test.add_section("Sachsen")
aufgabe = sektion.add_task("Nachbarn")

laender = aufgabe.response(4)
staaten = aufgabe.response(2)

aufgabe.item_body(
    "Das Bundesland Sachsen grenzt an " + laender
    + " andere deutsche Bundesländer und " + staaten
    + " Länder außerhalb Deutschlands.")

test.get_settings().set_answer_acc("exact")
test.create_test("output")
```

PySeA: Vorteil gegenüber der GUI

In der ONYX-Oberfläche

Bilder

mindestens 6 Klicks je Bild

Feedback

mehr als 7 Klicks je Aufgabe; Übergabe in Unteraufgaben umständlich und unübersichtlich

Variantensteuerung

aufwändig und unübersichtlich, Werte über viele Dialoge verteilt

Laufende Tests

Fehler in einer Lösung kann nicht geändert werden (Neustart für alle Studierenden nötig)

Mit PySeA

➤ `bilder = test.load_images("bilder").map()
aufgabe.item_body("Skizze:\n" + bilder["skizze.png"])`
Ordner einmal laden, im Text per Dateiname platzieren
(auch in Antwortoptionen und im Feedback)

➤ `aufgabe.feedback("incorrect", "Vorzeichen prüfen.", 2)`
Eine Zeile je Feedback: richtig oder falsch, ab Versuch n oder für einen Versuchsbereich (mit Bild und Formel)

➤ `test.variants("student_id_to_variant_assignment")
werte = sektion.add_excel_variables()`
Variante je Matrikelnummer, Werte aus einer Excel-Tabelle; Bilder und Tabellen wechseln je Variante mit.

➤ Lösungsabfrage funktioniert anders, daher Änderungen im Nachhinein möglich → kein Neustart nötig

PySeA Use Case: alles in Python

1. Die Möglichkeiten von Python nutzen

- **Erstellung von Bildern** (z.B. Funktionsverläufe in matplotlib)
- **Berechnung mit SymPy / NumPy / sStatics (Mechanik)**
- **Steuerung über Varianten**

2. Skriptbasierte Aufgaben mit Hilfe von KI erstellen lassen

- Grundgerüst kann von der KI gebaut werden → **Fokus auf den Inhalt und nicht die Umsetzung**
- Feedback automatisch erzeugen lassen → Grundlage ist erstmal vorhanden und kann final überarbeitet werden
- Automatische Erstellung von Übungsaufgaben für Studierende
 - Verwendung mathematischer Bibliotheken (SymPy, NumPy, ...) lässt Formeln leicht überprüfen
 - Mechanikaufgaben: Berechnung über sStatics (nur Input kommt von der KI)

Workshop PySeA

Voraussetzungen

- Python 3.12 + pip
- Git (oder Repo als Zip herunterladen)
- IDE (VS Code, PyCharm, ...)

Inhalt

- https://github.com/i4s-htwk/PySeA-public/tree/main/examples/PySeA_workshop
 - Minimalbeispiel: Einfache Teststruktur mit einer Rechenaufgabe
 - Feedback & Bilder: Erweiterung um Bilder und Feedback für richtige/falsche Antworten
 - Tabellen & Excel: Tabellen aus Excel-Dateien einbinden
 - Varianten: Variantenabhängige Aufgaben mit Excel-Daten